Written Assignment

DLMAIRIL01 – Reinforcement Learning

IU International University of Applied Sciences

Degree program: Master of Science Applied Artificial Intelligence

Learning and Search in AlphaGo and AlphaGo Zero

Gökhan Sen

Matriculation number: [to complete]

Supervisor/Tutor: [to complete]

Submission date: [to complete]

Contents

List of figures ..... **III**
List of tables ..... **III**
List of abbreviations ..... **III**
1 Introduction **1**
2 Foundations **2**
2.1 Go as a two-player zero-sum game of perfect information 2
2.2 Monte Carlo Tree Search and bandit-based selection 3
3 AlphaGo: learned priors and evaluations inside MCTS **4**
3.1 Network inventory and training pipeline 4
3.2 From imitation to reinforcement learning 5
3.3 Value network and leaf evaluation 6
3.4 Integration into search and component contribution 6
4 AlphaGo Zero: search as a policy-improvement operator **7**
4.1 Dual-head residual network and minimal input 7
4.2 Self-play loop and training targets 8
4.3 Search without rollouts and exploration mechanisms 9
5 Comparison and critical evaluation **10**
5.1 From Fan via Lee and Master to Zero 10
5.2 Attributing the performance gain 11
5.3 Limits: knowledge, compute, reproducibility and robustness 12
6 Conclusion **13**
References **15**
Appendix A: Statement on the use of AI-assisted tools **16**

List of figures

Fig. 1 Learning and search in AlphaGo Fan and AlphaGo Zero.....3

List of tables

Tab. 1 Network inventory and operational roles.....4

Tab. 2 Evolution and evidence boundaries across AlphaGo versions.....10

List of abbreviations

AI	Artificial intelligence
CNN	Convolutional neural network
CPU	Central processing unit
GPU	Graphics processing unit
KGS	KGS Go Server
MCTS	Monte Carlo Tree Search
MSE	Mean squared error
PUCT	Predictor-guided Upper Confidence bounds applied to Trees
RL	Reinforcement learning
SL	Supervised learning
TPU	Tensor Processing Unit
UCT	Upper Confidence bounds applied to Trees

1 Introduction

Go exposes a central challenge in artificial intelligence (AI). A typical branching factor of approximately 250 and games of around 150 moves make exhaustive enumeration impractical. Evaluating a nonterminal position is also difficult (Silver et al., 2016, p. 484). Monte Carlo tree search (MCTS) already provided a way to explore promising lines of play in Go (Browne et al., 2012, pp. 1, 27). AlphaGo substantially improved full-board play by using learned policies to guide that exploration and learned values to evaluate the positions it reached (Silver et al., 2016, pp. 484 –485).

AlphaGo Fan defeated Fan Hui 5–0 without handicap in October 2015; AlphaGo Lee subsequently defeated Lee Sedol 4–1, and AlphaGo Master achieved 60–0 against top professionals online (Silver et al., 2016, p. 488; Silver et al., 2017, p. 354, Methods section “AlphaGo versions”). The original study described defeating a professional on the full board as a feat previously expected to remain a decade away (Silver et al., 2016, p. 488). This substantiates the contemporary surprise, while distinguishing professional-level success from the later, stronger evidence of superhuman performance. The open analytical question is which interacting changes explain that progression.

The guiding research question is: How are deep neural networks integrated with Monte Carlo Tree Search in AlphaGo, and which published comparisons substantiate the changes introduced by AlphaGo Zero? Three questions organise the analysis: (1) Which networks are trained for which roles? (2) How do learned priors and values interact with search? (3) Which performance differences can be attributed to controlled design changes, and which remain confounded? The thesis is that search is both an inference-time improvement mechanism and, in Zero, a generator of training targets; architectural and resource changes also contribute materially (Silver et al., 2016, pp. 484–488; Silver et al., 2017, pp. 354–358).

Unit 5.2 of the course book introduced model-based reinforcement learning and MCTS and provided the starting point for this review. The review compares the original AlphaGo studies with foundational work on MCTS, policy gradients and residual networks, as well as selected independent analyses of reproducibility and robustness. The 2016 and 2017 studies supply primary performance and ablation evidence; other publications are used where they clarify a method or test the broader claims. Evidence is classified as (i) component ablations, (ii) fixed-data architectural tests, (iii) comparisons of training procedures, or (iv) competitive matches. These designs answer different causal questions: a reported win rate is not by itself evidence that a single changed component caused the improvement. This is a purposive, not exhaustive, source selection; neither a systematic database search nor experimental replication is claimed (Silver et al., 2016, pp. 484–488; Silver et al., 2017, pp. 354–358). The use of AI-assisted tools during preparation is described in Appendix A.

Unless a later variant is explicitly named, AlphaGo refers to AlphaGo Fan (2016), and AlphaGo Zero to the self-play system reported in 2017. The discussion of Lee and Master is used to qualify comparisons rather than to introduce separate research questions. Chapters 2–4 explain the decision problem, original training pipeline and Zero's feedback loop; Chapter 5 weighs the experimental evidence and limitations. The analysis uses published measurements and does not claim to independently reproduce the systems (Silver et al., 2016, pp. 484–488; Silver et al., 2017, pp. 354–358). The 2017 citations use the journal page numbers (354–359) and named Methods sections for details supplied after the main article.

2 Foundations

The distinction between learning a reusable decision function and searching a particular position underpins the comparison. The following sections establish the game formulation and the allocation of a finite search budget.

2.1 Go as a two-player zero-sum game of perfect information

Go is naturally formulated as an alternating, two-player Markov game with deterministic transitions and perfect information. The state includes the player to move and rule-relevant history; legal placements and pass are actions. An exact simulator implements transitions and terminal scoring. The players' objectives are opposed, with outcomes +1 and −1 from the respective player's perspective. Perfect information describes the game, whereas a chosen neural input can still omit information relevant to particular rules (Silver et al., 2016, Methods section “Problem setting”; Silver et al., 2017, Methods sections “Domain knowledge” and “Neural network architecture”).

A policy p(a|s) distributes probability across legal actions. Its value function estimates the expected return obtained from a state under that policy, rather than mechanically identifying the best move (Sutton & Barto, 2018, pp. 58–59). AlphaGo's signed outcome convention means that a value near zero represents approximately balanced expected outcomes under the represented play. A win for Black receives opposite signs when evaluated from Black's and White's perspectives. Tree backups must preserve that convention at successive alternating nodes (Silver et al., 2016, p. 486, Methods section “Search algorithm”).

Go supplies exact rules and terminal scoring, but not an affordable solution to choosing optimal actions. The simulator determines what happens after a move; a policy ranks candidate moves and a value function estimates the outcome of positions that cannot be searched to completion. Exact dynamics therefore do not eliminate the difficulty caused by branching and depth (Silver et al., 2016, p. 484; Sutton & Barto, 2018, pp. 159–161).

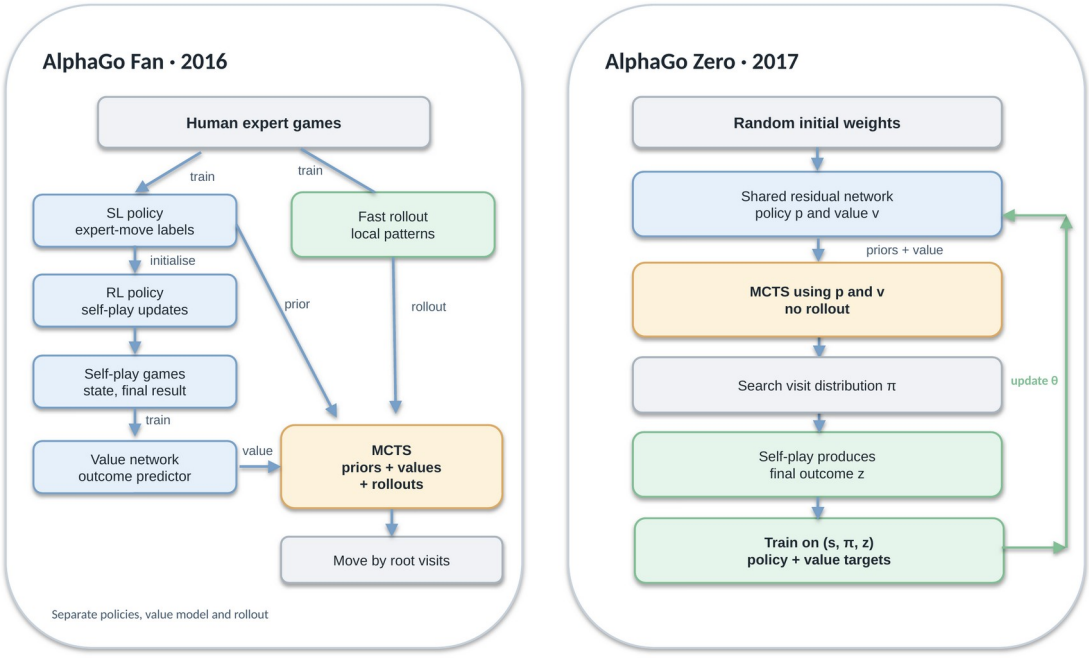
2.2 Monte Carlo Tree Search and bandit-based selection

MCTS allocates computation incrementally to a partial search tree. The standard cycle is selection, expansion, leaf evaluation and backup. Selection follows promising edges; expansion adds an encountered position; evaluation estimates its outcome, either through a continuation or a learned value; and backup updates statistics for the selected path (Browne et al., 2012, pp. 5–6). Fig. 1 places this common cycle alongside the different learning pipelines of AlphaGo Fan and AlphaGo Zero.

Tree search balances exploitation of favourable estimates against exploration of less-tested actions. UCT adapts confidence-guided bandit selection to tree planning (Kocsis & Szepesvári, 2006, Section 2). Prior-guided selection additionally uses an external predictor to distribute initial search attention, as analysed by Rosin (2011, p. 203). AlphaGo adapts this principle using policy-network probabilities, while edge values and visit statistics can outweigh a strong initial prior (Silver et al., 2016, Methods section “Search algorithm”; Silver et al., 2017, Methods section “Search algorithm”).

A network generalises across positions, whereas search spends its budget on one current position. Backpropagation alters network parameters during training; a tree backup alters local value and visit estimates during planning. Consequently, a search decision can disagree with the network’s preferred move even while all weights remain fixed. AlphaGo instantiates this division with a policy prior, a learned outcome estimate and a fast rollout. Its advantage over earlier MCTS systems depended not on search alone, but on making selective evaluation more informative within the available budget (Browne et al., 2012, p. 27; Silver et al., 2016, pp. 484–486).

Fig. 1 Learning and search in AlphaGo Fan and AlphaGo Zero



Source: Own representation based on Silver et al. (2016, pp. 484–486) and Silver et al. (2017, pp. 354–355).

3 AlphaGo: learned priors and evaluations inside MCTS

3.1 Network inventory and training pipeline

AlphaGo Fan assigns distinct objectives to three deep models and one lightweight policy. The supervised-learning (SL) policy predicts expert actions; its reinforcement-learning (RL) successor optimises self-play outcomes; the value network predicts outcomes under the RL policy. The fast rollout policy instead uses a linear softmax over local patterns. Tab. 1 makes the network-to-task correspondence explicit and includes Zero's later replacement for comparison (Silver et al., 2016, pp. 484–486, Methods section “Neural network architecture”; Silver et al., 2017, Methods section “Neural network architecture”).

Tab. 1 Network inventory and operational roles

Model	Input / architecture	Training data	Objective	Role in agent
Fan SL policy	48 planes; 13-layer CNN (match: 192 filters)	28.4 M KGS training positions (29.4 M total)	Predict expert moves	Prior in MCTS
Fan RL policy	SL-initialised convolutional policy	Self-play vs earlier policies	Improve signed outcome (policy gradient)	Generate value-training games
Fan value	CNN; dense layer; tanh scalar	>30 M states, one per diversified self-play game	Predict terminal outcome (MSE)	Leaf value estimate
Fan rollout	Local-pattern linear softmax; not deep	≈8 M human positions (Tygem)	Predict moves rapidly	Fast terminal rollouts
Zero dual-head	17 input planes; shared 20/40-block residual network	Search-guided self-play (s, π, z)	Policy cross-entropy + value MSE + regularisation	MCTS prior and leaf value; updated iteratively

Source: Own synthesis of Silver et al. (2016, pp. 484–486, Methods sections “Neural network architecture” and “Features for policy/value network”) and Silver et al. (2017, Methods section “Neural network architecture”).

The original supervised policy uses 48 feature planes on a 19 × 19 grid, encoding stone colour, liberties, captures, legality, move age and ladder-related tactics; the value model adds a plane indicating the player to move. The match policy has 13 convolutional layers with 192 filters in its hidden layers and a final softmax over board intersections. Extended Data Table 3 distinguishes that match configuration (55.4% test accuracy with one orientation) from the highest reported 57.0% accuracy, obtained with a 256-filter, eight-symmetry ensemble. Higher move-prediction accuracy is therefore not interchangeable with the precise architecture and latency of the match agent (Silver et al., 2016, p. 485, Methods sections “Features for policy/value network” and “Neural network architecture”, Extended Data Tables 2–3).

Human demonstrations supply the initial policy targets. Of the 29.4 million KGS positions, 28.4 million were used for training and one million for testing. The lightweight rollout policy was trained separately on about eight million Tygem positions. It predicted human moves with 24.2% accuracy and selected a move in about 2 microseconds. The deep policy took about 3 milliseconds per move (Silver et al., 2016, p. 485, Methods sections “Policy network: classification” and “Rollout policy”). The comparison explains why Fan could afford to use the fast policy to play out continuations and reserve deep policy evaluations for the search tree.

3.2 From imitation to reinforcement learning

For a demonstrated action a* in state s, the negative log-likelihood loss in Equation 1 is an explanatory single-example form of the original supervised objective. Minimisation increases the probability assigned to that expert action. The target establishes agreement with human play; it supplies dense, useful feedback before the system has learned to generate strong games for itself (Silver et al., 2016, pp. 484–485).

$$L_{SL} = -\log p_{\sigma}(a^* \mid s) \quad (1)$$

The RL policy starts from the SL parameters but replaces demonstration matching with outcome-weighted policy-gradient updates. Equation 2 expresses the main-text update, where ρ denotes policy parameters and zt is the final result from the player-to-move perspective. Positive outcomes reinforce sampled actions; negative outcomes reverse the direction. Williams (1992, Sections 4–5) establishes the likelihood-gradient principle of REINFORCE. In the detailed AlphaGo implementation, a baseline is also subtracted for variance reduction (initially zero, and in a later pass the value-network estimate). Thus Equation 2 is the simplified main-text rule, not a full transcription of the Methods implementation (Silver et al., 2016, p. 485, Methods section “Policy network: reinforcement learning”).

$$\Delta\rho \propto z_t \nabla_{\rho} \log p_{\rho}(a_t \mid s_t) \quad (2)$$

Without search, the RL policy wins more than 80% against its SL initialisation and 85% against Pachi using 100,000 simulations per move in this direct-policy comparison. The separate five-second-per-move tournament uses another Pachi configuration, reported at 400,000 simulations in Extended Data Table 6. During policy training, opponents are sampled from earlier versions to stabilise learning. These results support improvement in direct play, but comparisons across the two Pachi settings must not be conflated (Silver et al., 2016, p. 485, Extended Data Table 6).

The stronger standalone RL policy does not automatically make a stronger search prior. Although it improves direct-play results, the original system obtains better MCTS performance with the SL policy. A plausible reason is functional rather than contradictory: a policy trained to win may concentrate probability on fewer actions, whereas search benefits from a prior that exposes several promising alternatives for evaluation. The experiment establishes the direction of the

comparison, but does not isolate this proposed explanation as its cause (Silver et al., 2016, pp. 486–487). The result demonstrates why component quality must be evaluated in the role it actually performs.

3.3 Value network and leaf evaluation

The value network approximates v under policy pρ at state s, the expected signed outcome under the RL policy playing both sides, rather than a solved minimax value. Its tanh output lies between −1 and +1. Equation 3 gives the single-example squared-error objective used to fit completed self-play outcomes. The policy therefore tells search where to look, while the value function estimates the consequences of stopping a continuation early (Silver et al., 2016, p. 486).

$$L_{value} = \left(z - v_\theta(s) \right)^2 \quad (3)$$

Constructing the value targets is central to their reliability. Training from many correlated positions in complete human games produced mean squared errors of 0.19 on training data and 0.37 on test data. The replacement uses over 30 million positions, one sampled from each distinct self-play game, and reduces the corresponding gap to 0.226 versus 0.234. Importantly, the Methods describe how each game is generated: a prefix sampled from the SL policy, one random legal move to diversify the position, and a continuation under the RL policy; only one subsequent state and outcome is retained. This is not simply a random collection of all states from standard RL-policy games. Because the sampling scheme and generating policies change together, the observed generalisation improvement does not isolate a single cause (Silver et al., 2016, p. 486, Methods section “Value network: regression”).

At a leaf, AlphaGo combines a value-network prediction with the terminal result of a fast rollout as in Equation 4. Here sL is the leaf state, zL the rollout outcome and λ its mixing weight. The estimators describe outcomes under different policies, so they are complementary rather than interchangeable measurements. The paper reports that one value-network evaluation approaches the accuracy of rollouts driven by the stronger RL policy while using 15,000 times less computation; this is a comparison under the reported evaluation protocol, not a claim that every complete MCTS search becomes 15,000 times faster (Silver et al., 2016, p. 486, Figure 2b).

$$V(s_L) = (1 - \lambda)\, v_\theta(s_L) + \lambda\, z_L \quad (4)$$

3.4 Integration into search and component contribution

Each search edge stores a prior, a visit count and an action-value estimate. Selection trades off estimated value against prior-weighted exploration; expansion installs SL probabilities. Leaf evaluation combines the value network and rollout, and backup incorporates the evidence into traversed edges. Fan uses asynchronous CPU search and GPU network evaluation, with separate statistics for neural evaluations and rollouts. A temporary pattern-based tree policy supplies

guidance pending deep-policy expansion. The final move is the most visited root action (Silver et al., 2016, pp. 486–487, Methods section “Search algorithm”).

Search changes a recommendation through repeated local evidence, not through online neural-network training. A move initially favoured by the policy receives an exploration advantage, but its estimated value and visit statistics change as additional branches are evaluated. The root action with the most visits can therefore differ from the network’s highest-probability action. This explains the distinction between the prior and the final decision; it does not imply that every additional simulation is beneficial or that search is exact (Silver et al., 2016, Methods section “Search algorithm”).

The component ablations support the use of both learned values and rollouts. Without rollouts, the value-network-plus-MCTS variant still outperformed the other Go programs tested. The complete agent with mixed leaf evaluation ($\lambda = 0.5$) was strongest, winning at least 95% against the other AlphaGo variants (Silver et al., 2016, p. 488, Figure 4b, Extended Data Tables 7 and 9). Removing the learned prior, using only rollouts for leaf evaluation, and playing directly from the network also affected strength in distinct ways. These results show why “value-only” leaf evaluation and play without search are different tests. The gains from combining the components are demonstrated under the reported five-second, single-machine setting.

In the five-second-per-move tournament, single-machine AlphaGo wins 494 of 495 games against the comparison programs; distributed AlphaGo wins 77% against the single-machine version. The single-machine match uses 48 CPUs and eight GPUs, versus 1,202 CPUs and 176 GPUs for the distributed configuration. The original scalability experiments further vary search threads and GPU counts (Extended Data Table 8), indicating that engineering throughput is itself a measured contributor. These results are not a hardware-neutral comparison of neural-network algorithms, and the human Fan Hui match used longer time controls (Silver et al., 2016, pp. 487–488, Methods section “Evaluation”, Extended Data Tables 6 and 8).

Fan used its networks to improve decisions during play. Lee began using search-guided games to train its value network (Silver et al., 2017, Methods section “AlphaGo versions”). Zero extends that feedback to the policy as well. It trains on visit counts from search and outcomes from search-guided games.

4 AlphaGo Zero: search as a policy-improvement operator

4.1 Dual-head residual network and minimal input

Zero replaces the separate models with one shared residual tower and two heads. Its 17 binary input planes encode the current and seven preceding board positions for both players (16 planes), plus one colour-to-play plane. The history provides information relevant to repetition rules. The colour plane accounts for komi, the points awarded to White to compensate for moving second.

The simulator independently enforces legal transitions. Handcrafted tactical features are removed (Silver et al., 2017, Methods sections “Domain knowledge” and “Neural network architecture”).

The initial convolution and residual blocks use 256 filters. Equation 5 shows the residual connection before the final activation. The block learns a transformation added to its input, which makes deep networks easier to optimise (He et al., 2016, pp. 771–772). Zero’s blocks additionally use normalisation and rectified activations. Its 20- and 40-block configurations comprise an initial convolutional block and 19 or 39 residual blocks; each residual block contains two convolutions. Comparing these block counts to Fan’s layer count without qualification would be misleading (Silver et al., 2017, Methods section “Neural network architecture”).

$$y = x + F(x) \quad (5)$$

The policy head produces 362 action logits, one for each intersection and pass; the value head uses a 256-unit hidden layer and a scalar tanh output. With outcomes coded as ±1, a signed value estimate v corresponds to an estimated win probability of (v + 1)/2. These outputs share internal features but retain distinct targets and operational roles (Silver et al., 2017, Methods section “Neural network architecture”).

The smaller, 20-block configuration generated 4.9 million self-play games in three days, whereas the separate 40-block configuration generated 29 million games in 40 days. These are separate runs with different capacities and training durations. Sharing and residual depth therefore require their own evidence rather than attribution from the final headline score. Section 5.2 examines the fixed-data architecture ablation; first, the training loop explains what the shared network learns (Silver et al., 2017, pp. 355–356, 358).

4.2 Self-play loop and training targets

For each position, the network predicts (p, v), MCTS generates a search distribution π, and completed self-play supplies the signed outcome z from the perspective of the player to move in that position. The tuple (s, π, z) enters the replay data. Equation 6 combines value regression, cross-entropy against the search policy and parameter regularisation. The originality lies in the source of those targets: search-guided experience supplies both outputs, starting from random parameters rather than expert labels (Silver et al., 2017, pp. 354–355).

$$(p, v) = f_\theta(s), L = (z - v)^2 - \sum_a \pi_a \log p_a + c \| \theta \|^2 \quad (6)$$

Zero closes the loop between planning and learning. At decision time, its current network proposes priors and values and MCTS allocates simulations accordingly. At training time, the resulting root visit distribution π becomes a supervised target for the policy head, while the eventual game outcome z trains the value head. The network is subsequently updated and used to guide new searches. Thus the target is not simply the move a player happened to choose; it represents a

distribution refined by search under the current computational budget (Silver et al., 2017, pp. 354–355).

Calling this process policy improvement describes its intended direction, not an exact theoretical guarantee. Finite-budget MCTS may misrank moves, search statistics depend on exploration settings, and neural fitting can imperfectly reproduce the target distribution. Nevertheless, the repeated coupling can transfer information acquired by expensive local planning into the network's reusable predictions. The relevant empirical question is whether that feedback yields stronger play under controlled search and training conditions, not whether every self-play update must be better than its predecessor (Silver et al., 2017, pp. 354–355, Methods section "Reinforcement learning"; Sutton & Barto, 2018, pp. 447–450).

4.3 Search without rollouts and exploration mechanisms

Zero's search terminates a simulation at a leaf. At a new nonterminal leaf, one network call supplies both priors and value. A terminal state is scored using the game rules. Expansion and evaluation are coupled, followed by backup through the selected path. Rollouts disappear, but repeated model-based lookahead remains. Compared with Fan, new nonterminal leaves are expanded immediately, each thread waits for its neural evaluation, and the temporary tree policy is removed. The larger run additionally uses a transposition table (Silver et al., 2017, Methods section "Search algorithm").

Selection maximises Q + U, where Equation 7 specifies the prior-weighted exploration term. P is the network prior, N is the edge visit count, and c_{PUCT} is an exploration coefficient. The underlying prior-guided bandit idea predates AlphaGo (Rosin, 2011, p. 203). Fan's search also combines a prior term with action values, although its asynchronous rollout/value implementation differs. Zero keeps prior-guided selection while changing leaf evaluation and training targets (Silver et al., 2016, Methods section "Search algorithm"; Silver et al., 2017, Methods section "Search algorithm").

$$U(s,a) = c_{puct} P(s,a) \frac{\sqrt{\sum_b N(s,b)}}{1 + N(s,a)} \quad (7)$$

Computed illustration of Equation 7 (not a reported AlphaGo position): let c_{PUCT} = 1, total parent visits = 100, P(A) = 0.70, P(B) = 0.30, N(A) = 40, N(B) = 60, Q(A) = 0.20 and Q(B) = 0.40. Then U(A) = 0.70 × 10/41 ≈ 0.171 and U(B) = 0.30 × 10/61 ≈ 0.049. The resulting selection scores Q + U are approximately 0.371 and 0.449: the next simulation follows edge B despite its lower prior.

The published self-play procedure uses 1,600 simulations per move (approximately 0.4 s in the reported setup). Moves are sampled proportionally to visit counts for the first 30 moves, with temperature τ = 1, and thereafter the most visited action is selected as τ approaches zero. Root priors mix 75% network probabilities with 25% Dirichlet noise, using concentration 0.03. These

settings diversify complete games as well as allocating exploration within individual searches (Silver et al., 2017, Methods section “Self-play training pipeline”).

A new checkpoint faced the incumbent in 400 games, with 1,600 simulations per move. It became the player that generated self-play games only if it won more than 55% of the games (Silver et al., 2017, Methods section “Self-play training pipeline”). This gate measures progress against one incumbent under a specified search budget.

5 Comparison and critical evaluation

5.1 From Fan via Lee and Master to Zero

The evolution gradually integrates search-guided self-play into learning. Fan's value targets come from RL-policy games; Lee's come from AlphaGo's own search-guided games, iterated several times. Lee also enlarges its policy and value trunks to 12 convolutional layers with 256 planes and changes inference hardware. Master already shares Zero's network architecture, RL algorithm and MCTS, but retains expert initialisation, handcrafted features and rollouts (Silver et al., 2017, Methods section “AlphaGo versions”).

Tab. 2 Evolution and evidence boundaries across AlphaGo versions

Dimension	Fan	Lee	Master	Zero
Training	Human SL → policy-gradient self-play	Human-data start; search-guided self-play value	Human initialisation; search-guided RL	Search-guided self-play from random weights
Networks	Separate CNN policy and value	Deeper separate CNNs	Shared residual network	Shared residual network (20/40 blocks)
Inputs	Engineered features	Engineered features	Engineered features	17 history/colour planes
Rollouts	Fast policy	Retained	Retained	Removed
Playing hardware	176 GPUs (distributed)	48 TPUs (distributed)	4 TPUs	4 TPUs
Elo (5 s/move)	3,144	3,739	4,858	5,185 (40-day run)
Distinct matches	5–0 vs Fan Hui	4–1 vs Lee Sedol	60–0 online	100–0 vs Lee (3 days); 89–11 vs Master (40 days)

Source: Own synthesis based on Silver et al. (2016, pp. 487–488) and Silver et al. (2017, pp. 356–358, Methods sections “AlphaGo versions” and “Evaluation”).

Tab. 2 separates competitive results from training and inference conditions. The Elo ratings come from a tournament with five seconds per move. In separate matches, the three-day Zero beat AlphaGo Lee 100–0, and the 40-day Zero beat AlphaGo Master 89–11. Each player received two hours of main time plus three 60-second byoyomi periods. Fan’s 5–0 came from a different human-match protocol, and Master’s 60–0 came from an online series. These outcomes jointly establish progression, but their raw scores do not provide interchangeable effect sizes (Silver et al., 2016, Methods section “Evaluation”; Silver et al., 2017, pp. 356, 358, Methods section “Evaluation”).

The Master comparison is informative because Master and Zero use related residual architectures and search-guided reinforcement learning. However, they are not identical controlled treatments: feature representations, initialisation and rollout evaluation also differ. The result therefore supports

the effectiveness of Zero as a combined design, not a numerical estimate of the isolated effect of removing human examples (Silver et al., 2017, Methods section “AlphaGo versions”).

Three days and 40 days describe two published Zero training runs, whereas Lee was developed over several months. The timing and hardware descriptions are not phase-complete or normalised, so they do not establish a comparable end-to-end cost or energy advantage across systems (Silver et al., 2017, pp. 356, 358; Tian et al., 2019, pp. 6246–6247).

5.2 Attributing the performance gain

The architecture experiment trains four network designs on the same Zero-generated data and evaluates them with Zero’s search. Residual architecture adds over 600 Elo, and sharing the policy and value network adds approximately another 600. Sharing slightly reduces human-move prediction accuracy while improving value error and playing strength (Silver et al., 2017, pp. 356–357, Methods section “Neural network architecture comparison”). Each player has five seconds per move. The difference can therefore reflect the quality of the predictions and how many simulations each architecture makes possible in that time. The reported gains apply to this evaluation setup.

The target-source comparison holds architecture constant. The human-trained network predicts expert moves better and initially plays more strongly, but the self-play-trained player defeats it within the first 24 hours. The supervised comparator reaches 60.4% accuracy on the KGS test set and 54.3% on GoKifu professional-game validation; these different evaluation datasets must not be confused. This separates imitation accuracy from competitive improvement and supports the value of search-generated experience. It is nevertheless a partial control: the supervised comparator uses one-hot human move targets and downweights the value-loss term to 0.01, whereas self-play uses equally weighted losses. Experience volume and encountered positions differ too (Silver et al., 2017, p. 356, Methods section “Supervised learning”, Extended Data Table 1).

Master supplies a second, less controlled comparison. Zero's final rating is 5,185 against Master's 4,858, and it wins their head-to-head match 89–11. Since their architecture, RL algorithm and MCTS are shared, this 327-point rating difference supports the effectiveness of Zero's remaining design bundle. Assigning it entirely to removal of human data would confound that change with minimal features and rollout removal. Lee's earlier use of search-generated value targets further weakens a simple before-and-after story (Silver et al., 2017, p. 358, Methods section “AlphaGo versions”).

The strongest evidence for decision-time planning compares the final raw network with the complete Zero agent: 3,055 versus 5,185 Elo, a 2,130-point difference on the same tournament scale. It measures the additional decision-time effect of search on a network that was itself trained from search-generated targets; it cannot separately isolate the value of search during training. Nor is an Elo difference a linear multiple of intelligence or efficiency. The scale is anchored through

program and human references, to reduce inflation from self-play comparisons (Silver et al., 2017, p. 358, Methods section “Evaluation”).

The fixed-data experiment changes architecture while holding training data and the time budget constant. The raw-network comparison instead holds learned weights fixed and tests the benefit of search at decision time. The human-trained comparator holds architecture fixed but also changes the learning targets and loss weighting, while Master versus Zero changes initialisation, input features and rollouts together (Silver et al., 2017, pp. 356–358, Methods sections “Supervised learning”, “Neural network architecture comparison” and “AlphaGo versions”).

Taken together, the comparisons show that network design and decision-time search both contribute to playing strength. Fan’s ablations also show that learned values and rollouts can complement one another (Silver et al., 2016, pp. 486–488). Zero adds a way to feed the results of search back into training. The available experiments leave the independent effect of eliminating expert data unresolved.

5.3 Limits: knowledge, compute, reproducibility and robustness

The phrase “without human knowledge” must be read operationally. Zero removes human game demonstrations and the original handcrafted tactical input planes, but it retains legal move generation, perfect game rules and scoring, board-history observations, rotational and reflectional symmetries, and human-selected choices about architecture and search. This is not learning without any human-designed structure; it is learning without human move labels or handcrafted Go strategy features. The narrower interpretation follows directly from the specified inputs and training procedure (Silver et al., 2017, Methods section “Domain knowledge”).

Efficiency requires phase-specific accounting. Zero and Master play on four TPUs, versus Fan's 176 GPUs and Lee's 48 TPUs, yet Zero's network optimisation uses 64 GPU workers and 19 CPU parameter servers. Those figures do not constitute a complete accounting of the computational expense of self-play data generation and training. Tian et al. (2019, pp. 6246–6247) independently identify this reporting gap. Accelerator generations, utilisation, training duration and energy are not normalised, so the evidence establishes a smaller playing deployment, not a verified total-cost multiplier (Silver et al., 2017, p. 358, Methods section “Self-play training pipeline”).

Independent reimplementation strengthens the credibility of the approach while exposing practical variation. ELF OpenGo publishes code, models and self-play data; its professional match record is 20–0. Importantly, that record belongs to a prototype trained with a changing hybrid process, played at approximately 80,000 search simulations per move. Its fully documented main run is distinct. This supports reproducibility of superhuman performance in the algorithm family, rather than exact reproduction of every DeepMind ablation or of the prototype's training history (Tian et al., 2019, pp. 6244, 6247).

Competitive strength also leaves exploitable blind spots. Wang et al. (2023, pp. 35655–35658) train adversaries that win more than 97% against KataGo at superhuman settings by provoking blunders, despite losing to amateur humans. This tests a successor rather than the original Zero, under a targeted adversarial threat model. It nonetheless shows why ordinary tournament Elo can miss nontransitive weaknesses. Robustness evaluation should therefore include diverse opponents and deliberately unusual positions alongside average-case matches.

Transfer beyond Go requires three preconditions to be examined independently: a simulator accurate enough to support planning, enough computational budget to explore useful alternatives, and an evaluation protocol that detects failures outside routine trajectories. The first condition is unusually favourable in Go, with deterministic rules and legal moves. In a physical system, model error may be amplified when planning repeatedly uses incorrect predicted transitions; Sutton and Barto (2018, Section 8.3, pp. 166–168) discuss this problem. Consequently, the justified engineering recommendation is to validate search under perturbations of model dynamics and opponent or environment behaviour before interpreting self-play performance as reliable real-world control.

6 Conclusion

The first question concerned which networks serve which roles. In AlphaGo Fan, the supervised policy guides search, the RL policy generates experience for training the value network, and the value network and fast rollout policy evaluate leaves. Zero uses one shared network to provide both the policy prior and the value estimate (Silver et al., 2016, pp. 485–488; Silver et al., 2017, pp. 354–355). These roles explain why a policy that plays better on its own can still be less effective at guiding search.

The second question concerned how learning and search interact. The prior guides exploration, while leaf evaluations update action values and visit counts that can overturn the network’s initial preference. In Zero, the search visit distribution becomes the policy target, and the eventual game outcome trains the value head (Silver et al., 2016, pp. 486–488; Silver et al., 2017, pp. 354–355). Search therefore improves decisions locally and supplies training signals that can influence decisions in other positions.

The third question concerned which changes the evidence can isolate. Fixed-data tests show a benefit from residual layers and a shared network under the reported search conditions. The comparison with Zero’s raw network isolates the benefit of adding search at decision time. The Master–Zero results show an advantage for the full design, with expert-data initialisation, features and rollouts changing together (Silver et al., 2017, pp. 356–358, Methods section “AlphaGo versions”). The size of the gain attributable to removing expert examples remains open.

The analysis has two main limitations. It is a purposive review without reproduced training runs, and published systems differ in architectures, resources and evaluation protocols. Consequently, neither win rates nor training duration alone establish the marginal causal effect or efficiency of one component. Beyond Go, learned values and planning are only as reliable as the simulator, observations and evaluated operating conditions; model bias can accumulate through repeated search (Sutton & Barto, 2018, pp. 166–168). Robustness tests of Go agents further caution against equating exceptional average strength with immunity to targeted failures (Wang et al., 2023, pp. 35655–35658).

A follow-up experiment could vary expert-data initialisation, handcrafted features and rollout use independently. Network capacity, training experience and search simulations per move would be held constant. Reporting playing strength, outcome calibration and computational cost against both familiar and unfamiliar opponents would show whether gains transfer beyond the original benchmark. For applied reinforcement learning, search-generated targets are promising where the simulator is accurate and the search budget is adequate. Where dynamics are uncertain, test how planning responds to model error and unfamiliar trajectories before relying on its decisions.

References

Browne, C. B., Powley, E., Whitehouse, D., Lucas, S. M., Cowling, P. I., Rohlfshagen, P., Tavener, S., Perez, D., Samothrakis, S., & Colton, S. (2012). A survey of Monte Carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games, 4*(1), 1–43. https://doi.org/10.1109/TCIAIG.2012.2186810

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 770–778). IEEE. https://doi.org/10.1109/CVPR.2016.90

Kocsis, L., & Szepesvári, C. (2006). Bandit based Monte-Carlo planning. In J. Fürnkranz, T. Scheffer, & M. Spiliopoulou (Eds.), *Machine learning: ECML 2006* (pp. 282–293). Springer. https://doi.org/10.1007/11871842_29

Rosin, C. D. (2011). Multi-armed bandits with episode context. *Annals of Mathematics and Artificial Intelligence, 61*(3), 203–230. https://doi.org/10.1007/s10472-011-9258-6

Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T., Leach, M., Kavukcuoglu, K., Graepel, T., & Hassabis, D. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature, 529*(7587), 484–489. https://doi.org/10.1038/nature16961

Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., Chen, Y., Lillicrap, T., Hui, F., Sifre, L., van den Driessche, G., Graepel, T., & Hassabis, D. (2017). Mastering the game of Go without human knowledge. *Nature, 550*(7676), 354–359. https://doi.org/10.1038/nature24270

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.

Tian, Y., Ma, J., Gong, Q., Sengupta, S., Chen, Z., Pinkerton, J., & Zitnick, C. L. (2019). ELF OpenGo: An analysis and open reimplementation of AlphaZero. In K. Chaudhuri & R. Salakhutdinov (Eds.), *Proceedings of the 36th International Conference on Machine Learning* (pp. 6244–6253). PMLR. https://proceedings.mlr.press/v97/tian19a.html

Wang, T. T., Gleave, A., Tseng, T., Pelrine, K., Belrose, N., Miller, J., Dennis, M. D., Duan, Y., Pogrebniak, V., Levine, S., & Russell, S. (2023). Adversarial policies beat superhuman Go AIs. In A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, & J. Scarlett (Eds.), *Proceedings of the 40th International Conference on Machine Learning* (pp. 35655–35739). PMLR. https://proceedings.mlr.press/v202/wang23g.html

Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning, 8*(3–4), 229–256. https://doi.org/10.1007/BF00992696

Appendix A: Statement on the use of AI-assisted tools

I used ChatGPT (OpenAI) as a learning and writing aid while preparing this assignment. I used it to discuss and clarify reinforcement learning concepts relevant to AlphaGo and AlphaGo Zero, including the role of neural networks and Monte Carlo tree search. I also used it to review drafts, suggest revisions, and improve the clarity and precision of the wording.

The technical claims in this assignment are supported by the sources cited in the main text. I take responsibility for the final analysis, wording, and references.